

南开大学

汇编语言与逆向技术课程实验报告

实验四：ARM 平台-HelloWorld



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

1 实验时间

第 11 周星期一 9-10 节

2 实验目的

1. 理解 GNU ARM 汇编代码运行环境的搭建、配置及编译运行，掌握在华为鲲鹏云服务器上进行环境配置
2. 命令行输出 “HelloWorld”

3 实验环境

华为鲲鹏云主机、openEuler20.03 操作系统

4 实验步骤

4.1 创建 hello 目录

```
1 mkdir hello
2 cd hello
```

4.2 创建示例程序代码 hello.s

```
1 vim hello.s
```

在出来的页面里输入代码，输入完后通过 `esc` 回到命令模式，输入英文冒号和 `wq` 保存并退出。

```
[root@ecs-0112 ~]# mkdir hello
[root@ecs-0112 ~]# cd hello
[root@ecs-0112 hello]# vim hello.s
```

4.3 进行编译运行

```
1 as hello.s -o hello.o
2 ld hello.o -o hello
3 ./hello
```

5 汇编语句的解析

5.1 完整代码及注释：

```
1 .text      @ 代码段开始，存放程序执行指令
2 .global _start @ 声明_start为全局符号，作为程序入口点
3 _start:    @ 程序开始执行的地方
4     mov x0,#0    @ x0 = 0，设置文件描述符为标准输出(stdout)
5     ldr x1,msg    @ x1 = 字符串msg的内存地址
6     mov x2,len    @ x2 = 字符串长度len的值
7     mov x8,64    @ x8 = 64，设置系统调用号为64(write系统调用)
8     svc #0      @ 执行软中断，进入内核模式调用系统调用
9     mov x0,123    @ x0 = 123，设置程序退出状态码为123
10    mov x8,93     @ x8 = 93，设置系统调用号为93(exit系统调用)
11    svc #0      @ 执行软中断，调用exit系统调用结束程序
12 .data      @ 数据段开始，存放初始化数据
13 msg:       @ 字符串标签msg
14     .ascii "Hello World!\n " @ 定义ASCII字符串"Hello World!"后跟换行符和空格
15 len=.-msg    @ 计算字符串长度：当前位置减去msg地址，得到字节长度
```

5.2 部分代码解释：

5.2.1 系统调用 write

```
1 mov x0,#0
2 ldr x1,msg
3 mov x2,len
4 mov x8,64
5 svc #0
```

这部分代码调用 write 系统调用向标准输出打印字符串。svc # 0 触发软中断，进入内核模式执行系统调用。

5.2.2 系统调用 exit

```
1 mov x0,#0
2 ldr x1,msg
3 mov x2,len
4 mov x8,64
5 svc #0
```

这部分代码调用 exit 系统调用结束程序，程序将以状态码 123 退出。

5.2.3 字符串定义

```
1 msg:
2     .ascii "Hello World!\n "
```

定义以 msg 为标签的字符串，包含“Hello World!” 后跟换行符和空格。

6 运行示例截图

```
[root@ecs-0112 hello]# as hello.s -o hello.o
[root@ecs-0112 hello]# ld hello.o -o hello
[root@ecs-0112 hello]# ./hello
Hello World!
[root@ecs-0112 hello]#
```

图 6.1: Hello World 结果示例

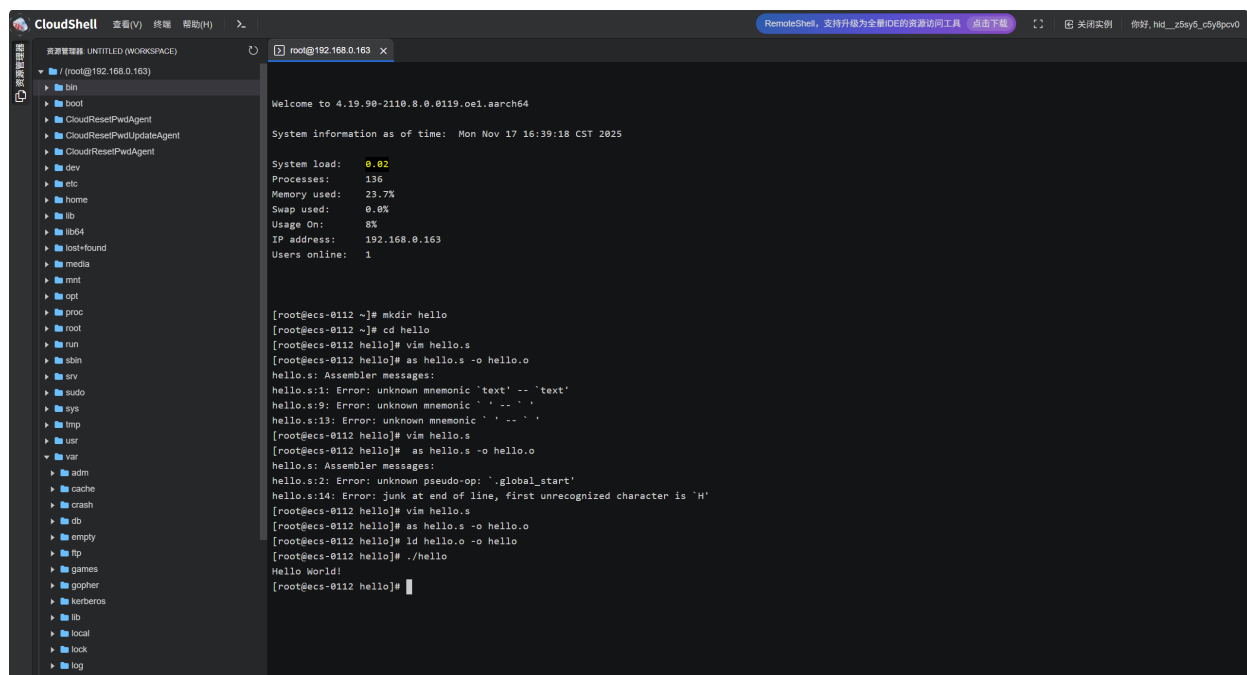


图 6.2: 全流程展示

7 问题回答

同样的代码能否在 x86 平台运行，为什么？

答：不能。

1. 二进制不兼容: ARM 和 x86 的机器指令编码完全不同。
2. 寄存器架构: 寄存器名称、大小和使用约定不同。ARM64: x0-x30 这些 64 位通用寄存器；x86: eax, ebx, ecx, edx 等 32 位寄存器。

3. 系统调用机制: 系统调用号、参数传递方式、中断指令都不同。
4. 汇编器语法: 即使源代码相似, 也需要不同的汇编器来编译。